
System Design: Job Scheduling System

Distributed Task Scheduler — Senior Engineer Interview Walkthrough

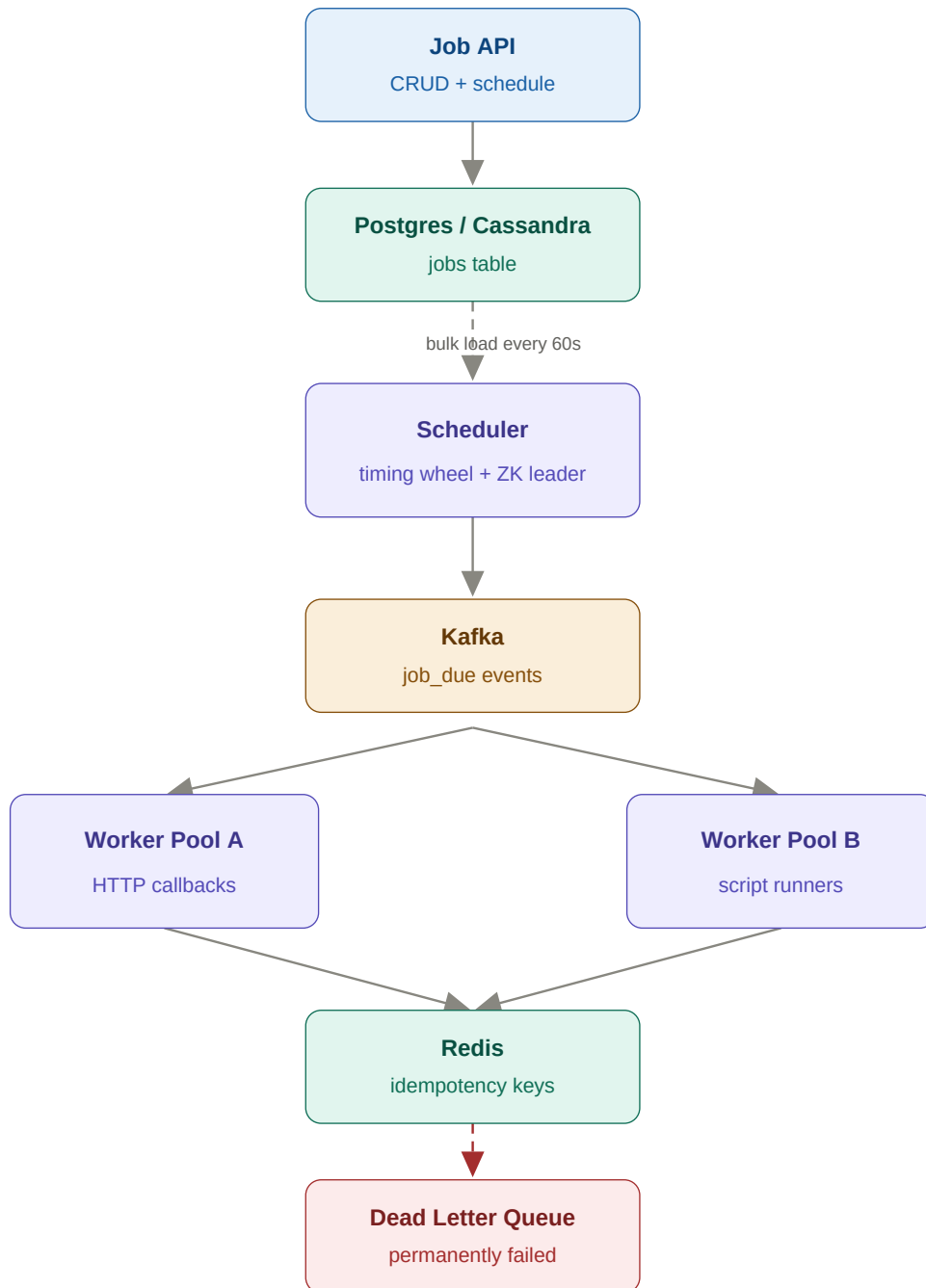
1. Requirements & Capacity

Think out loud: "One-time or recurring? At-least-once or exactly-once? Max delay? I'll assume: both, at-least-once + idempotency, < 5s delay, 1M jobs/day at peak."

Scale	Users/RPS	Architecture	Storage
Low	1k jobs/day	Single scheduler + Postgres + 1 worker	<1 GB
Mid	100k/day	HA scheduler pair + Postgres + Redis + worker pool	~10 GB
High	1M/day ~12 RPS avg	Timing wheel + Kafka + Cassandra + autoscale workers	~500 GB

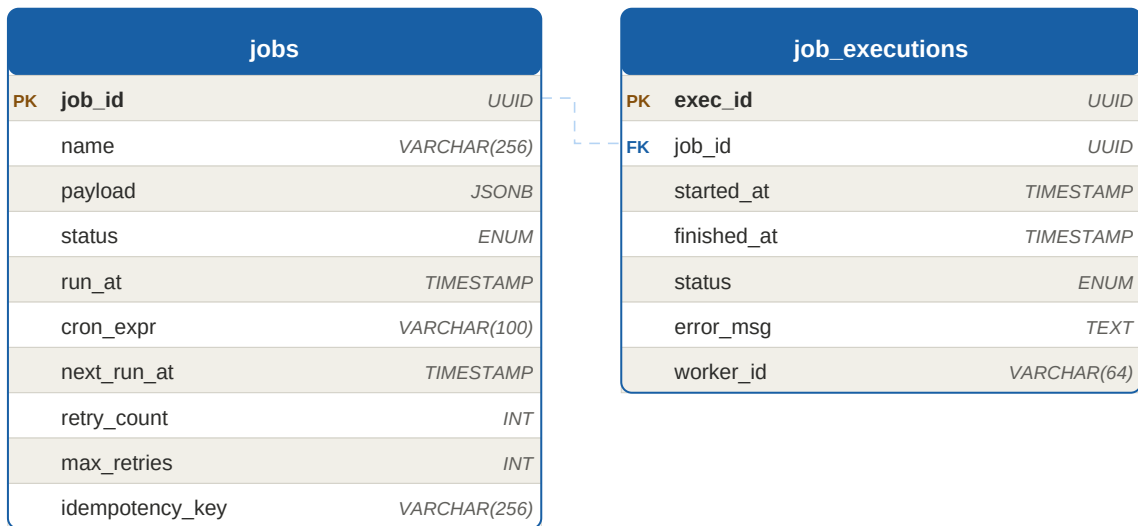
2. Architecture Diagram

Think out loud: "The key design question: how does the scheduler know which jobs are due? Naive DB poll is $O(N)$. Timing wheel gives $O(1)$ dispatch. Let me draw the high-scale version."



High-scale architecture: timing wheel pre-loads jobs from DB every 60s. Workers check Redis for idempotency before executing.

3. DB Schema



Why this choice: Index on (status, run_at) makes the polling query fast: `SELECT * FROM jobs WHERE status='PENDING' AND run_at <= NOW()`. At low/mid scale this single index handles the load. At high scale we replace polling with the timing wheel.

Why idempotency_key: idempotency_key has a UNIQUE constraint. If the scheduler re-dispatches a job after a crash, the worker checks Redis for this key before executing. Duplicate execution is prevented even across worker crashes.

4. API & Trade-offs

POST	/api/v1/jobs	{name, payload, run_at, cron_expr?, max_retries?}
GET	/api/v1/jobs/{job_id}	Status + execution history
DELETE	/api/v1/jobs/{job_id}	Cancel if PENDING
GET	/api/v1/jobs?status=FAILED&limit;=50	List by status

Decision	Choice	Gain	Trade-off
Delivery	At-least-once	Simple, resilient	Workers must be idempotent
Dispatch	Timing wheel	O(1) per tick, no poll storm	In-memory state, needs WAL
Queue	Kafka	Replay, partition-per-type	Ops complexity
Leader elect	ZooKeeper	No split-brain	ZK dependency